Production-ready Spring Boot applications

3rd semester @ Erhvervsakademi København

GitHub

Outline

- `sdkman` for managing Java versions
- `JavaDocs` for documenting your code
- `OpenAPI` for documenting your REST APIs
- `Logging` with `SLF4J` and `Logback`
- `Spring Boot Actuator` for monitoring and management
- API versioning strategies
- JPA performance tips
- Database migrations with `Flyway`
- Code quality and design patterns

Questions about your fullstack project?

`sdkman` for managing Java versions

> Java has many versions and you may need different versions for different projects. `sdkman` is a tool that makes it easy to install and switch between Java versions.

Installation (Mac/Linux):

```
curl -s "https://get.sdkman.io" | bash
```

> Widows users can use WSL to run `sdkman` in a Linux environment. See sdkman installation instructions for details.

`sdkman` usage - installing and switching Java versions

List available Java versions:

```
sdk list java
```

Install a specific Java version:

```
sdk install java 25.0.2-tem
```

Switch to a different Java version (it is needs to be installed first)**:**

```
sdk use java 17.0.2-tem
```

`sdkman` environment variables

> Create a `.sdkmanrc` file in your project root to specify the Java version for that project:

```
sdk env init
```

This creates a `.sdkmanrc` file with the current Java version.

Use the project-specific Java version:

```
sdk env
```

If not installed, use:

```
sdk env install
```

JavaDocs for documenting your code

```java
/**
 * This class represents a user in the system.
 */
public class User {
    /**
     * Constructs a new User with the given username and email.
     *
     * @param username the username of the user
     * @param email the email address of the user
     */
    public User(String username, String email) {}
}
```

> Use `/** ... */` to create JavaDoc comments. You can include tags like `@param` and `@return` for method parameters and return values. You can also add HTML tags for formatting.

OpenAPI for documenting your REST APIs

> OpenAPI (formerly Swagger) is a specification for documenting REST APIs. You can use the `springdoc-openapi` library to automatically generate OpenAPI documentation for your Spring Boot application.

Add the dependency:

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
</dependency>
```

> This will generate OpenAPI documentation for all your REST controllers and provide a Swagger UI at `/swagger-ui.html` and the OpenAPI JSON at `/v3/api-docs`.

OpenAPI annotations

```java
@Operation(summary = "Get all users")
@GetMapping
public List<User> getAllUsers() {
    // ...
}

@Operation(summary = "Create a new user")
@Parameter(name = "user", description = "The user to create", required = true)
@PostMapping
public User createUser(@RequestBody User user) {
    // ...
}
```

> Use `@Operation` to add a summary and description to your API endpoints. You can also use other annotations like `@Parameter` to document method parameters and `@ApiResponse` to document possible responses.

Logging with SLF4J and Logback

> SLF4J is a logging facade that allows you to use different logging implementations (like Logback) without changing your code.

The dependency is already included in Spring Boot Starter, so you can use it right away:

```
public class UserService {
    private static final Logger logger = LoggerFactory.getLogger(UserService.class);

    public void createUser(User user) {
        logger.info("Creating user: {}", user.getUsername());
        // ...
    }
}
```

Logging levels

- `TRACE` — Very detailed logs, typically only useful for debugging.
- `DEBUG` — Detailed logs for debugging purposes.
- `INFO` — General informational messages about the application's progress.
- `WARN` — Potentially harmful situations or important events that are not errors.
- `ERROR` — Error events that might still allow the application to continue running.

You can configure the logging level in `application.properties`:

```
logging.level.root=INFO
logging.level.com.example=DEBUG
```

Spring Boot Actuator for monitoring and management

> Spring Boot Actuator provides production-ready features to help you monitor and manage your application. It includes endpoints for health checks, metrics, and more.

Add the dependency:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

> This will enable several endpoints under `/actuator`, such as `/actuator/health` for health checks and `/actuator/metrics` for application metrics.

Customizing Actuator endpoints

> By default, only a few Actuator endpoints are exposed. You can customize which endpoints are available by setting `management.endpoints.web.exposure.include` in your `application.properties`.

```
management.endpoints.web.exposure.include=health,info
```

Actuator - info with Git information

> Sometimes it's difficult to figure out which version of the application is running in production.

Use the `git-commit-id-plugin` to generate a `git.properties` file with Git information:

```
<plugins>
    <plugin>
        <groupId>io.github.git-commit-id</groupId>
        <artifactId>git-commit-id-maven-plugin</artifactId>
    </plugin>
</plugins>
```

Then the git information will be available at `/actuator/info`:

> You need to run `./mvnw -DskipTests clean package` to generate the `git.properties` file before running the application.

API versioning strategies - Header versioning

> Spring Boot 4 makes it easy to version your REST APIs. You can use URI versioning or header versioning.

Configuration for header versioning:

```
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void configureApiVersioning(ApiVersionConfigurer configurer) {
        configurer.useRequestHeader("API-Version");
        configurer.setDefaultVersion("1.0");
        configurer.setVersionRequired(false);
    }
}
```

> **Usage:** Include a request header: `API-Version: 1.0` or `API-Version: 2.0`

API versioning strategies - URI versioning

Configuration for URL versioning:

```
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void configureApiVersioning(ApiVersionConfigurer configurer) {
        configurer.usePathSegment(1);
        configurer.setDefaultVersion("1.0");
        configurer.setVersionRequired(false);
    }
}
```

API versioning - usage in RestController

URI versioning:

```
@RestController
@RequestMapping("/api/v{version}/users")
public class UserControllerV1 {
    @GetMapping(version = "1")
    public List<User> getAllUsers() {}

    @GetMapping(version = "2")
    public List<User> getAllUsersV2() {}
}
```

This will make the endpoint available at `/api/v1/users` and `/api/v2/users`.

Spring Data JPA performance tips

> A common issue is the `n+1` query problem, where fetching a list of entities causes additional queries to fetch related entities.

Solution: Use `@EntityGraph` to fetch related entities in a single query:

```
public interface UserRepository extends JpaRepository<User, Long> {
    @EntityGraph(attributePaths = "posts")
    @Query("SELECT u FROM User u")
    List<User> findAllWithPosts();
}
```

> This will fetch users and their posts in a single query, improving performance.

Database migrations

> In a production environment, you can't just drop the database and start over when you make changes to the schema. You need a way to manage database migrations.

> Solution: Use a database migration tool like `Flyway` to manage your database schema changes.

Setup Flyway

> In IntelliJ, you need to add the `Flyway` and `JPA Buddy` plugin to your project.

- Go to `File > Settings > Plugins` and search for `Flyway` and `JPA Buddy`, then install both plugins. You may need to restart IntelliJ after installing the plugins.

Add the Flyway dependency (including MySQL support):

```
<dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-core</artifactId>
</dependency>
<dependency>
    <groupId><org.flywaydb</groupId>
    <artifactId>flyway-mysql</artifactId>
</dependency>>
```

Flyway configuration in `application.properties`

> **In a production environment, we need to ensure that JPA does not automatically create or update the database schema, as this can lead to data loss.**

```
spring.jpa.hibernate.ddl-auto=validate
spring.flyway.enabled=true
```

> This configuration tells Spring Boot to validate the database schema against the JPA entities and to enable Flyway for managing database migrations.

Flyway migration scripts

> Flyway migration scripts are SQL files that define the changes to be made to the database schema.

> They are placed in the `src/main/resources/db/migration` directory and must follow a specific naming convention: `V{version}__{description}.sql`.

This adds a `flyway_schema_history` table to the database to track which migrations have been applied.

Code quality examples from your project

```java
public BigDecimal calculate(List<String> seatTypes, int durationMinutes,
                            String movieFormat, int numberOfTickets) {
        BigDecimal total = BigDecimal.ZERO;

        for (String seat : seatTypes) {
            BigDecimal price = new BigDecimal("100.00");

            if (seat.equals("REGULAR")) {
                price = price.add(new BigDecimal("5.00"));
            } else if (seat.equals("SOFA")) {
                price = price.add(new BigDecimal("10.00"));
            }

            if (movieFormat.equals("THREE_D")) {
                price = price.add(new BigDecimal("20.00"));
            } else if (movieFormat.equals("IMAX")) {
                price = price.add(new BigDecimal("30.00"));
            }

            if (durationMinutes > 170) {
                price = price.add(new BigDecimal("15.00"));
            }

            total = total.add(price);
        }

        if (numberOfTickets <= 5) {
            total = total.add(total.multiply(new BigDecimal("0.07")));
        }

        if (numberOfTickets > 10) {
            total = total.subtract(total.multiply(new BigDecimal("0.05")));
        }

        return total;
    }
```

Code quality issues

> The above code has several issues:

- It uses hardcoded values for prices and thresholds, which makes it difficult to maintain and update.
- All rules live in a single method, which violates the Single Responsibility Principle and makes it hard to read and test.
- Adding a rule means modifying existing code, which can introduce bugs, and invalidate existing tests.
- Difficult to test each rule in isolation, leading to complex test cases that cover multiple rules at once.

Refactoring ideas

- Instead of mixing all rules in a single method, we can create separate classes for each rule that implement a common interface, e.g., `TicketPricingRule`.
- Introduce a `TicketPricingService` that takes a list of `TicketPricingRule` instances and applies them to calculate the total price.

```
public interface TicketPricingRule {
    void apply(TicketPriceRequest request, TicketPriceResult result);
}
```

Rule implementation

```java
@Component
@Order(1)  // specify the order of rule application
class RegularSeatRule implements TicketPricingRule {
    @Override
    public void apply(TicketPriceRequest request, TicketPriceResult result) {
        if (request.getSeatType().equals("REGULAR")) {
            result.addPrice(new BigDecimal("5.00"));
        }
    }
}
```

> Each rule is now encapsulated in its own class, making it easier to maintain and test. Adding a new rule simply means creating a new class that implements `TicketPricingRule` without modifying existing code.

Using `@Component` and `@Order`

> By annotating the rule classes with `@Component`, Spring will automatically detect and register them as beans. The `@Order` annotation specifies the order in which the rules should be applied.

Since we have multiple rules, we can inject a list of `TicketPricingRule` into the `TicketPricingService` and apply them in the specified order - based on the value in `@Order(1)`

Using the rules based design

```java
public class DefaultTicketPricingService implements TicketPricingService {
    private final List<TicketPricingRule> rules;

    public DefaultTicketPricingService(List<TicketPricingRule> rules) {
        this.rules = rules;
    }

    @Override
    public BigDecimal calculate(TicketPriceRequest request) {
        TicketPriceResult result = new TicketPriceResult();
        for (TicketPricingRule rule : rules) {
            rule.apply(request, result);
        }
        return result.getTotalPrice();
    }
}
```

Benefits of the rules-based design

- **Separation of concerns**: Each rule is responsible for a specific aspect of the pricing logic, making it easier to understand and maintain.
- **Open/Closed Principle**: The design is open for extension (adding new rules) but closed for modification (existing code does not need to be changed to add new rules).
- **Testability**: Each rule can be tested in isolation, leading to simpler and more focused test cases.
- **Flexibility**: The order of rule application can be easily changed by modifying the `@Order` annotation without changing the logic of the rules themselves.